

Mobile App Development Tutorial

Exit Exam Preparation, with Hands-on Code

Purpose: This handout summarizes the most exam-relevant Android basics and adds small practical code examples so students can connect theory with implementation.

1. Core Topics at a Glance

- **Activities**, they represent screens in an Android app.
- **Activity lifecycle**, it controls how an activity is created, shown, paused, stopped, and destroyed.
- **Intents**, they allow one component to request an action from another component.
- **Layouts** and **Views**, they define and display the user interface.
- **Resources**, such as strings, colors, and layouts, keep the app organized.
- **Fragments**, reusable user interface pieces hosted inside activities.
- **Services**, components used for background work.
- **Optimization** patterns, such as **ViewHolder**, improve smooth scrolling and performance.

2. Activity Basics

An activity is a single screen with which the user interacts. In most beginner questions, MainActivity is the entry screen of the app.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Remember: setContentView() connects the Java activity class to its XML layout file.

- If the activity class is MainActivity.java, the usual layout file is activity_main.xml.
- onCreate() is where basic initialization happens.
- The method startActivity() is used to open another activity when no return value is expected.

Mini Practice

Open a second screen when a button is clicked:

```
Button btnNext = findViewById(R.id.btnNext);

btnNext.setOnClickListener(v -> {
    Intent intent = new Intent(MainActivity.this, SecondActivity.class);
    startActivity(intent);
});
```

3. Activity Lifecycle

The lifecycle describes what happens as an activity moves between states.

Method	Meaning
onCreate()	Called when the activity is first created.
onStart()	Called when the activity becomes visible to the user.
onResume()	Called when the activity is ready for user interaction.
onPause()	Called when the activity is partially hidden.

onStop()	Called when the activity is no longer visible.
onDestroy()	Called before the activity is removed from memory.

Lifecycle Demo

```
@Override
protected void onStart() {
    super.onStart();
    Log.d("LIFECYCLE", "Activity is now visible");
}

@Override
protected void onResume() {
    super.onResume();
    Log.d("LIFECYCLE", "User can now interact");
}
```

4. Intents

An Intent is a messaging object used to request an action from another app component.

- Explicit Intent, opens a known class inside your app.
- Implicit Intent, requests a general action, such as opening a web page.

Explicit Intent Example

```
Intent intent = new Intent(this, ProfileActivity.class);
startActivity(intent);
```

Implicit Intent Example

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://www.google.com"));
startActivity(intent);
```

Passing Data with Intent

```
Intent intent = new Intent(this, WelcomeActivity.class);
intent.putExtra("username", "Abel");
startActivity(intent);
```

```
String name = getIntent().getStringExtra("username");
txtResult.setText("Welcome " + name);
```

5. XML Layouts and Views

Android user interfaces are usually designed in XML. The XML file contains views such as TextView, EditText, Button, and ImageView.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/txtTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```

        android:text="Student Login"
        android:textSize="22sp" />

<EditText
    android:id="@+id/edtName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name" />

<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit" />
</LinearLayout>

```

Connecting XML Views in Java

```

EditText edtName = findViewById(R.id.edtName);
Button btnSubmit = findViewById(R.id.btnSubmit);
TextView txtTitle = findViewById(R.id.txtTitle);

```

6. Input Validation

A very common exam area is checking whether an EditText is empty.

```

if (edtName.getText().length() == 0) {
    edtName.setError("Name is required");
} else {
    Toast.makeText(this, "Valid input", Toast.LENGTH_SHORT).show();
}

```

Common Trap: Do not compare EditText content using `== ""`. The standard beginner-safe way is `getText().length() == 0`.

7. Resources

Resources are stored separately from Java code. This improves organization and supports reuse.

- strings.xml stores text values
- colors.xml stores color values
- activity_main.xml stores layout structure

String Resource Example

```

<resources>
    <string name="app_name">MyApp</string>
    <string name="login">Login</string>
</resources>

```

```
txtTitle.setText(R.string.login);
```

8. Fragments

A fragment is a reusable part of the interface that lives inside an activity.

- Useful for modular design
- Can have its own layout and lifecycle

- A fragment lifecycle method often asked in exams is `onAttach()`

Simple Fragment Example

```
public class HomeFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_home, container, false);
    }

    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
    }
}
```

9. Services

A Service performs background operations without a user interface.

- Example uses: music playing, downloading data, syncing data
- In MCQs, Service is the correct component for background operations

Simple Service Example

```
public class MyService extends Service {

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        Log.d("SERVICE", "Service started");
        return START_STICKY;
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

Start the Service

```
Intent intent = new Intent(this, MyService.class);
startService(intent);
```

10. ListView, RecyclerView, and ViewHolder Pattern

The ViewHolder pattern caches view references so the app scrolls more smoothly. In exams, its primary purpose is improving scrolling performance by avoiding repeated view lookups.

```
static class ViewHolder {
    TextView txtName;
}

@Override
public View getView(int position, View convertView, ViewGroup parent) {
    ViewHolder holder;

    if (convertView == null) {
        convertView = LayoutInflater.from(context)
            .inflate(R.layout.item_user, parent, false);

        holder = new ViewHolder();
```

```

        holder.txtName = convertView.findViewById(R.id.txtName);
        convertView.setTag(holder);
    } else {
        holder = (ViewHolder) convertView.getTag();
    }

    holder.txtName.setText(userList.get(position));
    return convertView;
}

```

11. Toasts and User Feedback

Toast is a quick popup message used to inform the user.

```

Toast.makeText(this, "Login successful", Toast.LENGTH_SHORT).show();

```

12. AndroidManifest.xml

The AndroidManifest file registers app components and basic app information.

```

<application
    ... >
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <activity android:name=".SecondActivity" />
    <service android:name=".MyService" />
</application>

```

13. Short Hands-on Exercise

Build a small app with one EditText, one Button, and one TextView. When the button is clicked, validate the input and show the entered name.

```

EditText edtName = findViewById(R.id.edtName);
Button btnShow = findViewById(R.id.btnShow);
TextView txtOutput = findViewById(R.id.txtOutput);

btnShow.setOnClickListener(v -> {
    if (edtName.getText().length() == 0) {
        edtName.setError("Please enter your name");
    } else {
        String name = edtName.getText().toString();
        txtOutput.setText("Hello, " + name);
    }
});

```

What this exercise covers: findViewById, button click listener, EditText validation, string extraction, and TextView update.

14. Quick Exam Revision Sheet

Term	Meaning
startActivity()	Starts an activity when no result is expected.
activity_main.xml	Standard XML layout file for MainActivity.

onStart()	Called when the activity becomes visible.
Intent	Messaging object to request an action.
setContentView()	Sets the layout for an activity.
getText().length() == 0	Simple way to check whether EditText is empty.
Service	Used for background operations.
onAttach()	A fragment lifecycle method.
ViewHolder	Caches views for smooth scrolling.

15. Practice Questions, with Expected Answers

Q: Which method starts a new activity without expecting a returned value?

A: startActivity()

Q: Which file usually defines the layout of MainActivity?

A: activity_main.xml

Q: Which lifecycle method makes the activity visible?

A: onStart()

Q: What is an Intent?

A: A messaging object used to request an action from another component.

Q: Which component handles background work?

A: Service

16. Final Advice for Students

- Memorize the lifecycle order: onCreate, onStart, onResume, onPause, onStop, onDestroy.
- Practice writing simple XML layouts and connecting them with findViewById.
- Understand the difference between Activity, Fragment, Intent, and Service.
- Do not only memorize MCQ answers, also practice tiny code snippets.

View Based Activity + Java

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/hello_message_textview"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.502"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <Button
        android:id="@+id/greet_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Next"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/hello_message_textview"
        app:layout_constraintVertical_bias="0.386" />

    <Button
        android:id="@+id/google_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Google"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/greet_button"
        app:layout_constraintVertical_bias="0.17" />

    <Button
        android:id="@+id/welcome_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/google_button"
        app:layout_constraintVertical_bias="0.362" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.java

```
package com.thedreamerplc.training.android.jetpackcompose.exitexamone;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
```

```

import android.view.View;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        Button greetButton = findViewById(R.id.greet_button);
        greetButton.setOnClickListener(v -> greet());

        Button googleButton = findViewById(R.id.google_button);
        googleButton.setOnClickListener(v -> openGoogle());

        Button welcomeButton = findViewById(R.id.welcome_button);
        welcomeButton.setOnClickListener(v -> openWelcome());
    }

    private void greet() {
        Intent intent = new Intent(this, GreetingActivity.class);
        startActivity(intent);
    }

    private void openGoogle() {
        Intent intent = new Intent(Intent.ACTION_VIEW);
        intent.setData(Uri.parse("https://www.google.com"));
        startActivity(intent);
    }

    private void openWelcome() {
        Intent intent = new Intent(this, WelcoomeActivity.class);
        intent.putExtra("username", "Ribka");
        startActivity(intent);
    }

    @Override
    protected void onStart() {
        super.onStart();
        Log.d("LIFECYCLE", "Activity is now visible");
    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d("LIFECYCLE", "User can now interact");
    }
}

```

activity_greeting.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"

```



```

    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".GreetingActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Second Activity"
        android:textSize="24sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

GreetingActivity.java

```

package com.thedreamerplc.training.android.jetpackcompose.exitexamone;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class GreetingActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_greeting);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}

```

activity_welcome.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".WelcomeActivity">

    <TextView
        android:id="@+id/welcome_message"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text=""
        android:textSize="24sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

```

```

<EditText
    android:id="@+id/name_edit_text"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="text"
    android:hint="Name"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.497"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/welcome_message"
    app:layout_constraintVertical_bias="0.167" />

<Button
    android:id="@+id/check_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Check"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.498"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/name_edit_text"
    app:layout_constraintVertical_bias="0.38" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

WelcomeActivity.java

```

package com.thedreamerplc.training.android.jetpackcompose.exiteexamone;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class WelcomeActivity extends AppCompatActivity {

    private TextView welcomeMessage;
    private EditText nameEditText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_welcome);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        welcomeMessage = findViewById(R.id.welcome_message);

        String name = getIntent().getStringExtra("username");
        welcomeMessage.setText(getString(R.string.welcome, name, "Aloha"));

        nameEditText = findViewById(R.id.name_edit_text);
    }
}

```

```
Button checkButton = findViewById(R.id.check_button);
checkButton.setOnClickListener(v -> check());
}

private void check() {
    if (nameEditText.getText().toString().isEmpty()) {
        nameEditText.setText("Please enter your name");
    } else {
        Toast.makeText(this, "Valid Input", Toast.LENGTH_SHORT).show();
    }
}
}
```

strings.xml

```
<resources>
    <string name="app_name">ExitExamOne</string>
    <string name="welcome">"Welcome Again %1$s %2$s!"</string>
</resources>
```

Jetpack Compose + Kotlin

MainActivity.kt

```
package com.thedreamerplc.training.android.jetpackcompose.woof

import android.os.Bundle
import android.widget.Space
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.annotation.DrawableRes
import androidx.annotation.StringRes
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.animateContentSize
import androidx.compose.animation.core.Spring
import androidx.compose.animation.core.animate
import androidx.compose.animation.core.spring
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ExpandLess
import androidx.compose.material.icons.filled.ExpandMore
import androidx.compose.material3.Card
import androidx.compose.material3.CenterAlignedTopAppBar
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.dimensionResource
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.tooling.preview.Preview
import com.example.woof.data.Dog
import com.example.woof.data.dogs
import com.thedreamerplc.training.android.jetpackcompose.woof.ui.theme.WoofTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            WoofTheme {
                // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize()
                ) {
                    WoofApp()
                }
            }
        }
    }
}
```

```

    }
}

/**
 * Composable that displays an app bar and a list of dogs.
 */
@Composable
fun WoofApp() {
    Scaffold(
        topBar = {
            WoofTopAppBar()
        }
    ) { it ->
        LazyColumn(contentPadding = it) {
            items(dogs) {
                DogItem(
                    dog = it,
                    modifier = Modifier.padding(dimensionResource(R.dimen.padding_small))
                )
            }
        }
    }
}

/**
 * Composable that displays a list item containing a dog icon and their information.
 */
@Composable
fun DogItem(
    dog: Dog,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    val color by animateColorAsState(
        targetValue = if (expanded) MaterialTheme.colorScheme.tertiaryContainer
        else MaterialTheme.colorScheme.primaryContainer,
    )

    Card(modifier = modifier) {
        Column(
            modifier = Modifier
                .animateContentSize(
                    animationSpec = spring(
                        dampingRatio = Spring.DampingRatioNoBouncy,
                        stiffness = Spring.StiffnessMedium
                    )
                )
                .background(color = color)
        ) {
            Row(
                modifier = modifier
                    .fillMaxWidth()
                    .padding(dimensionResource(R.dimen.padding_small))
            ) {
                DogIcon(dog.imageResourceId)
                DogInformation(dog.name, dog.age)
                Spacer(modifier = Modifier.weight(1f))
                DogItemButton(
                    expanded = expanded,
                    onClick = { expanded = !expanded }
                )
            }
            if (expanded) {
                DogHobby(
                    dog.hobbies,
                    modifier = Modifier.padding(
                        start = dimensionResource(R.dimen.padding_medium),
                        top = dimensionResource(R.dimen.padding_small),
                    )
                )
            }
        }
    }
}

```

```

        end = dimensionResource(R.dimen.padding_medium),
        bottom = dimensionResource(R.dimen.padding_medium)
    )
}
}
}
}

@Composable
private fun DogItemButton(
    expanded: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
){
    IconButton(
        onClick = onClick,
        modifier = modifier
    ){
        Icon(
            imageVector = if (expanded) Icons.Filled.ExpandLess else Icons.Filled.ExpandMore,
            contentDescription = stringResource(R.string.expand_button_content_description),
            tint = MaterialTheme.colorScheme.secondary
        )
    }
}

@Composable
fun DogHobby(
    @StringRes dogHobby: Int,
    modifier: Modifier = Modifier
){
    Column(
        modifier = modifier
    ){
        Text(
            text = stringResource(R.string.about),
            style = MaterialTheme.typography.labelSmall
        )
        Text(
            text = stringResource(dogHobby),
            style = MaterialTheme.typography.bodyLarge
        )
    }
}

/**
 * Composable that displays a photo of a dog.
 *
 * @param dogIcon is the resource ID for the image of the dog
 * @param modifier modifiers to set to this composable
 */
@Composable
fun DogIcon(
    @DrawableRes dogIcon: Int,
    modifier: Modifier = Modifier
){
    Image(
        modifier = modifier
            .size(dimensionResource(R.dimen.image_size))
            .padding(dimensionResource(R.dimen.padding_small))
            .clip(MaterialTheme.shapes.small),
        contentScale = ContentScale.Crop,
        painter = painterResource(dogIcon),

        // Content Description is not needed here - image is decorative, and setting a null content
        // description allows accessibility services to skip this element during navigation.

        contentDescription = null
    )
}

```

```

/**
 * Composable that displays a dog's name and age.
 *
 * @param dogName is the resource ID for the string of the dog's name
 * @param dogAge is the Int that represents the dog's age
 * @param modifier modifiers to set to this composable
 */
@Composable
fun DogInformation(
    @StringRes dogName: Int,
    dogAge: Int,
    modifier: Modifier = Modifier
){
    Column(modifier = modifier) {
        Text(
            text = stringResource(dogName),
            style = MaterialTheme.typography.displayMedium,
            modifier = Modifier.padding(top = dimensionResource(R.dimen.padding_small))
        )
        Text(
            text = stringResource(R.string.years_old, dogAge),
            style = MaterialTheme.typography.bodyLarge
        )
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun WoofTopAppBar(modifier: Modifier = Modifier) {
    CenterAlignedTopAppBar(
        title = {
            Row(
                verticalAlignment = Alignment.CenterVertically
            ){
                Image(
                    modifier = Modifier
                        .size(dimensionResource(R.dimen.image_size))
                        .padding(dimensionResource(R.dimen.padding_small)),
                    painter = painterResource(R.drawable.ic_woof_logo),
                    contentDescription = null
                )
                Text(
                    text = stringResource(R.string.app_name),
                    style = MaterialTheme.typography.displayLarge
                )
            }
        },
        modifier = modifier
    )
}

/**
 * Composable that displays what the UI of the app looks like in light theme in the design tab.
 */
@Preview
@Composable
fun WoofPreview() {
    WoofTheme(darkTheme = false) {
        WoofApp()
    }
}

```

Flutter + Dart

quote.dart

```
class Quote {  
  Quote({  
    required this.author,  
    required this.text,  
  });  
  
  String author;  
  String text;  
}
```

quote_card.dart

```
import 'package:flutter/material.dart';  
  
import 'quote.dart';  
  
class QuoteCard extends StatelessWidget {  
  const QuoteCard({  
    super.key,  
    required this.quote,  
    required this.deleteQuote,  
  });  
  
  final Quote quote;  
  final Function() deleteQuote;  
  
  @override  
  Widget build(BuildContext context) {  
    return Card(  
      margin: EdgeInsets.fromLTRB(16.0, 16.0, 16.0, 0.0),  
      color: Colors.amberAccent[200],  
      child: Padding(  
        padding: EdgeInsetsGeometry.all(12.0),  
        child: Column(  
          children: [  
            Text(  
              '${quote.text}',  
              style: TextStyle(  
                fontSize: 18.0,  
                fontWeight: FontWeight.bold,  
                color: Colors.grey[600],  
              ),  
            ),  
            SizedBox(height: 6.0),  
            Text(  
              '${quote.author}',  
              style: TextStyle(  
                fontSize: 14.0,  
                color: Colors.blue[800],  
              ),  
            ),  
            SizedBox(height: 8.0),  
            TextButton.icon(  
              onPressed: deleteQuote,  
              icon: Icon(Icons.delete),  
              label: Text('delete quote'),  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```


main.dart

```
import 'package:flutter/material.dart';

import 'quote.dart';
import 'quote_card.dart';

void main() {
  runApp(
    const MaterialApp(
      home: QuoteList(),
    ),
  );
}

class QuoteList extends StatefulWidget {
  const QuoteList({super.key});

  @override
  State<QuoteList> createState() => _QuoteListState();
}

class _QuoteListState extends State<QuoteList> {
  List<Quote> quotes = [
    Quote(
      author: 'Oscar Wilde',
      text: 'Be yourself; everyone else is already taken',
    ),
    Quote(
      author: 'Oscar Wilde',
      text: 'I have nothing to declare except my genius',
    ),
    Quote(
      author: 'Oscar Wilde',
      text: 'The truth is rarely pure and never simple',
    ),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.grey[200],
      appBar: AppBar(
        title: const Text('Awesome Quotes'),
        centerTitle: true,
        backgroundColor: Colors.redAccent,
      ),
      body: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: quotes
          .map(
            (quote) => QuoteCard(
              quote: quote,
              deleteQuote: () {
                setState(() {
                  quotes.remove(quote);
                });
              },
            ),
          )
          .toList(),
      ),
    );
  }
}
```